

Dado que Dart no admite herencia múltiple para evitar problemas como la duplicación de estado o la ambigüedad estructural (el problema del diamante), los *Mixins* ofrecen una forma elegante de reutilizar código en múltiples jerarquías de clases sin relación directa.

11.1 El Concepto de Mixin y la Palabra Reservada `with`

Un **Mixin** es una clase que encapsula propiedades y métodos diseñados para ser "mezclados" en otras clases mediante la palabra clave `with`. A diferencia de la herencia (`extends`), el objetivo de un mixin no es establecer una relación jerárquica de identidad ("es un"), sino dotar a una clase de una capacidad o comportamiento específico ("puede hacer" o "tiene la capacidad de").

- Se declara con la palabra reservada `mixin NombreMixin`.
- **No pueden ser instanciados** directamente ni poseen constructores.
- Una clase puede aplicar múltiples mixins separándolos por comas: `class MiClase extends SuperClase with MixinA, MixinB`.

```
mixin Volador {  
  void volar() => print('Elevándose por los aires...');  
}  
  
mixin Nadador {  
  void nadar() => print('Nadanado en el agua...');  
}  
  
// Composición de comportamientos múltiples  
class Pato extends Ave with Volador, Nadador {}
```

11.2 Restricciones de Mixins mediante la Cláusula `on`

Podemos restringir el uso de un mixin para que solo pueda ser aplicado a clases que hereden o implementen una superclase específica mediante la palabra clave `on`.

Esto garantiza que el mixin tenga acceso explícito a las propiedades y métodos de esa superclase concreta, lo cual resulta vital en Flutter al definir mixins sobre la clase base `State<T>` para el manejo de animaciones o ciclos de vida de interfaz.

```
abstract class DispositivoBase {  
  bool encendido = false;  
}  
  
// Este mixin SOLO puede aplicarse en clases que extiendan DispositivoBase  
mixin DiagnosticoElectronico on DispositivoBase {  
  void ejecutarTest() {  
    if (encendido) {  
      print('Ejecutando prueba de diagnóstico...');  
    }  
  }  
}
```

```
    } else {
      print('Error: El dispositivo debe estar encendido para probarlo.');
```

11.3 Mixins vs Herencia (**extends**) vs Interfaces (**implements**)

Para diseñar arquitecturas limpias y mantenibles en Dart y Flutter, debemos elegir la herramienta estructural adecuada:

Mecanismo	Sintaxis	¿Reutiliza Código?	Permite Estado/Atributos	Casos de Uso en Flutter
Herencia	extends	Sí	Sí (Única superclase)	Jerarquía base (StatelessWidget , StatefulWidget).
Interfaz	implements	NO (Solo contratos)	Sí (Obliga a redefinir)	Repositorios desacoplados, contratos de API.
Mixin	with	Sí (Múltiples mixins)	Sí (Campos y métodos)	Animaciones (TickerProviderStateMixin), persistencia de pestañas (KeepAlive).

11.4 Ejemplo

Creen el archivo `bin/d11_mixins_composicion.dart` en VS Code. El siguiente programa muestra cómo dotar a servicios de datos de capacidades transversales (logging de auditoría, encriptación y soporte de caché) mediante mixins.

```
/// Módulo D11: Mixins y Composición de Comportamientos en Dart
/// Demostración de reutilización de código mediante mixins, restricciones `on` y
composición múltiple.

void main() {
  print('=====');
  print('  SISTEMA DE SERVICIOS Y AUDITORÍA (D11 POO) - ESCOM      ');
  print('=====\\n');

  // 1. Instanciación del servicio de autenticación con comportamientos mezclados
  print('--- 1. Servicio de Autenticación (Logger + Encriptación) ---');
  final servicioAuth = ServicioAutenticacion();
  final String passHasheado = servicioAuth.hashPassword('Escom2026!');
  servicioAuth.registrarAcceso('Login exitoso del usuario OZZY');
  print('Hash generado: $passHasheado\\n');

  // 2. Instanciación del servicio de expediente con restricción `on`
  print('--- 2. Servicio de Expediente Académico (Con Caché y Logger) ---');
```

```

final servicioExpediente = ServicioExpedienteEscom();
servicioExpediente.conectar();
servicioExpediente.guardarEnCache('EXP-2026', 'Promedio: 9.2');
print('Consulta desde caché: ${servicioExpediente.obtenerDeCache('EXP-2026')}');

print('\n=====');
}

/// Mixin 1: Otorga capacidad de log de auditoría
mixin LoggerAuditoria {
  void registrarAcceso(String evento) {
    final hora = DateTime.now().toIso8601String().substring(11, 19);
    print(' [AUDITORÍA $hora]: $evento');
  }
}

/// Mixin 2: Otorga capacidad de encriptación de cadenas
mixin EncriptadorSeguro {
  String hashPassword(String textoPlano) {
    // Simulación de hashing
    return 'SHA256::${textoPlano.codeUnits.join('')}';
  }
}

/// Clase base abstracta para servicios que requieren conexión
abstract class ServicioConectable {
  bool conectado = false;

  void conectar() {
    conectado = true;
    print(' [CONEXIÓN] Servicio de datos inicializado.');
  }
}

/// Mixin 3 con restricción `on`: Solo aplicable a clases que extiendan
ServicioConectable
mixin SoporteCache on ServicioConectable {
  final Map<String, String> _cacheLocal = {};

  void guardarEnCache(String clave, String valor) {
    if (conectado) {
      _cacheLocal[clave] = valor;
      print(' [CACHE] Guardado exitoso -> Clave: $clave');
    } else {
      print(' [CACHE ERROR] Imposible cachear sin conexión activa.');
```

```
/// Servicio 2: Extiende ServicioConectable y combina SoporteCache y
LoggerAuditoria
class ServicioExpedienteEscom extends ServicioConectable with LoggerAuditoria,
SoporteCache {}
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d11_mixins_composicion.dart
```

11.5 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para consolidar la composición mediante mixins y el uso de la cláusula `on`.

Ejercicio 1: Sistema de Dispositivos Móviles IoT y Conectividad

Crea un modelo para sensores inteligentes de laboratorio:

1. Define los mixins:
 - **ConectableWifi**: Métodos `void conectarWifi(String red)` y `void desconectarWifi()`.
 - **ConectableBluetooth**: Métodos `void vincularBluetooth(String dispositivo)` y `void emitirSenal()`.
2. Crea una clase base **DispositivoLab** con el atributo `String id`.
3. Crea la clase **SensorMultiprotocolo** que extienda **DispositivoLab** y aplique ambos mixins (`with ConectableWifi, ConectableBluetooth`).
4. En `main()`, instancia un sensor, conéctalo a una red Wi-Fi y vincúlalo por Bluetooth mostrando sus mensajes en consola.

Ejercicio 2: Gestor de Estado de UI con Notificación de Cambios (Patrón Flutter)

Escribe un modelo simulado de controlador de vista:

1. Crea la clase base **ControladorVista** con la propiedad `bool cargando = false`.
2. Define un mixin **NotificadorCambios** con restricción `on ControladorVista`:
 - Implementa un método `void notificarUI(String accion)` que verifique si `cargando` es `true`. Si es así, imprime `'[UI] Renderizando pantalla en estado de carga...'`, de lo contrario imprime `'[UI] Actualizando interfaz tras la acción: $accion'`.
3. Crea la clase **PerfilUsuarioControlador** que extienda **ControladorVista** con **NotificadorCambios**.
4. Simula en `main()` el cambio de estado de carga y la notificación a la interfaz.